Introduzione alla Programmazione

Corso di Laurea in
Ingegneria Informatica e Automatica



Aspetti Organizzativi e Introduzione

Professori:
Antonella Poggi e Marco Schaerf (canale 1: A-L)
Giuseppe Santucci (canale 2: M-Z)

Prima lezione

- Aspetti organizzativi
- Descrizione del corso
- Introduzione ai linguaggi di programmazione e a Python
- Prime istruzioni in Python
- L’ambiente Spyder

Informazioni generali sul corso

9 CFU

Il corso è erogato nel primo semestre (23 Settembre 2026 – 22 Dicembre 2026)

Due canali:
Canale 1 (A-L) – proff. Antonella Poggi e Marco Schaerf
antonella.poggi@uniroma1.it
marco.schaerf@uniroma1.it
Canale 2 (M-Z) – prof. Giuseppe Santucci
giuseppe.santucci@uniroma1.it

Le lezioni si tengono in Aula 105, presso l'edificio Marco Polo, viale dello scalo San Lorenzo 82, nei giorni di lunedì e mercoledì e nell'Aula 204 dello stesso edificio il venerdì

L'attività di laboratorio si svolge il Martedì a partire dal 29 Settembre presso il laboratorio Paolo Ercoli via Tiburtina 205 (aule 15 & 16)

Orari Canale 1

- Lunedì 08.00 – 10.00
- Martedì 11.00 – 13.00 (Laboratorio aule 15 e 16)
- Mercoledì 10.00 – 12.00
- Venerdì 08.00 – 10.00

Orari Canale 2

- Lunedì 14.00 – 16.00
- Martedì 13.00 – 15.00 (Laboratorio aule 15 e 16)
- Mercoledì 14.00 – 16.00
- Venerdì 16.00 – 18.00

Esercitazioni presso il laboratorio Paolo Ercoli



Via Tiburtina, 205

Le esercitazioni in laboratorio si svolgono ogni martedì nella fascia oraria 11:00-15:00.

Sono organizzate in 2 turni in presenza

Prima Esercitazione: ***Martedì 29 settembre 2026 !!!!***

Il corso in estrema sintesi

- Prerequisiti: nessuno (veramente)
- I contenuti del corso sono divisi in due parti:
  - Programmazione: programmi scritti in un linguaggio eseguibile da un computer. Nello specifico, il linguaggio Python
  - Teoria: "Modelli dell'informatica" : l'informatica non è solo programmazione

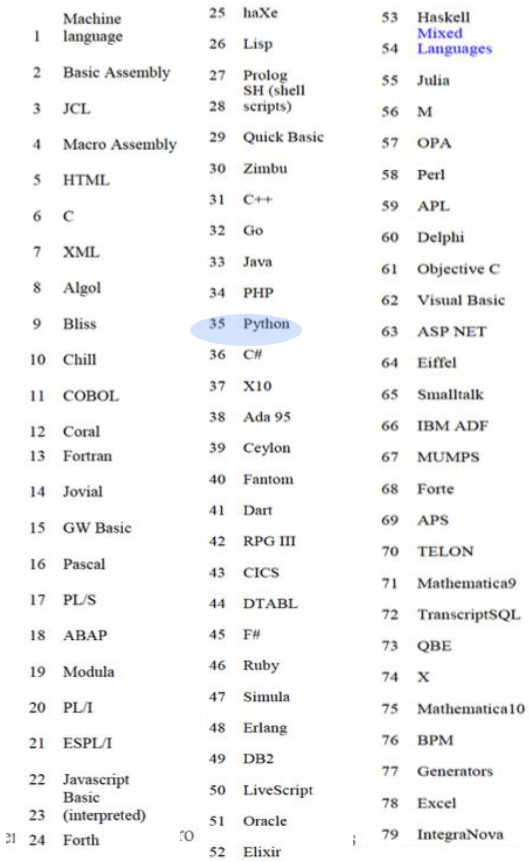
Il corso non è facile (anche per chi sa già programmare):

- ***Impegnarsi e studiare fin dall'inizio*** *(anche se non ci sono interrogazioni in classe)*
- ***Esercitarsi nella programmazione*** *(Casa-Laboratorio)*

Una domanda

- Perché Python?
- Nei prossimi corsi:
  - C
  - Java
  - Assembler
  - ...

Perché tutti questi linguaggi?



...

Programma: Programmazione

- Nozioni introduttive: Il calcolatore. Algoritmi e Programmi. Linguaggi di programmazione. La compilazione
- Nozioni elementari sulla programmazione (con esempi in Python): I/O di base. Uso dell'ambiente di sviluppo Spyder
- Aspetti di base della programmazione (con esempi in Python): Espressioni aritmetiche e tipi di dati elementari. Variabili e istruzioni di assegnazione. Il tipo di dato stringa
- Decisioni: costrutti if ed else-if (elif)
- Cicli: Ciclo for. Ciclo while. Cicli annidati
- Funzioni e moduli: Introduzione alla programmazione con funzioni. Moduli e loro uso. Esecuzione delle funzioni
- Liste: Proprietà di base. Operazioni sulle liste. Algoritmi elementari che fanno uso di liste. Rappresentazione di tabelle e matrici
- Dizionari: Proprietà di base. Accesso e manipolazione di dizionari
- File e file system: Apertura, chiusura e manipolazione di file di testo. Funzioni di base per l'accesso al file system
- Elementi di programmazione in C: accenni ed esempi

Programma: Modelli dell'Informatica

- Architettura dei calcolatori: Architettura di von Neumann. Esempi di Linguaggio Macchina. La legge di Moore

- Rappresentazione dell'Informazione: Rappresentazione di caratteri e stringhe, Rappresentazione dei numeri positivi. Rappresentazione in complemento a due. Rappresentazione in virgola mobile. Errore assoluto ed errore relativo

Appelli d’esame

- 2 Appelli a gennaio e febbraio 2027
- 2 Appelli a giugno e luglio 2027
- 1 Appello a settembre 2027

Modalità d'esame

- La prova d'esame consiste di
  - **Una prova pratica al calcolatore**, durante la quale lo studente dovrà realizzare alcune funzioni in **Python** e rispondere a delle **domande di teoria**. La durata dell'esame è di circa 1h e 45m

- La prova è identica e si svolge nello stesso giorno per i due canali

- Durante il corso saranno mostrati degli esempi di prove di esame

Materiale Didattico

- Per la parte di Programmazione
  - **Slide e dispense del corso** (distribuite sul sito)
  - **Testi e soluzioni delle esercitazioni**
  - Libri di Riferimento (non obbligatori):

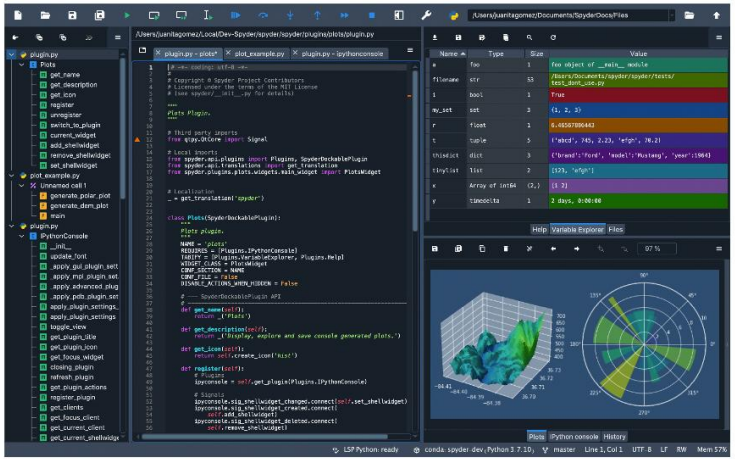
    Horstmann, Necaise "Concetti di Informatica e fondamenti di Python" (versione breve), (Testo più semplice, adatto a chi non ha mai programmato)

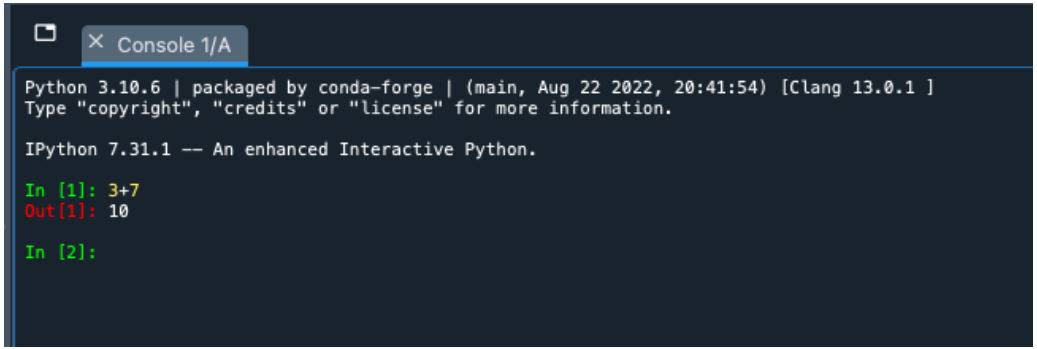
    John V. Guttag “Introduzione Alla Programmazione Con Python: Dal Pensiero Computazionale Al Machine Learning”, (testo più avanzato, adatto a chi sa già programmare)
- Per la parte di Modelli
  - **Slide e dispense** (distribuite sul sito)

Ambiente di Lavoro per la Programmazione

– Ambiente integrato Spyder https://www.spyder-ide.org/

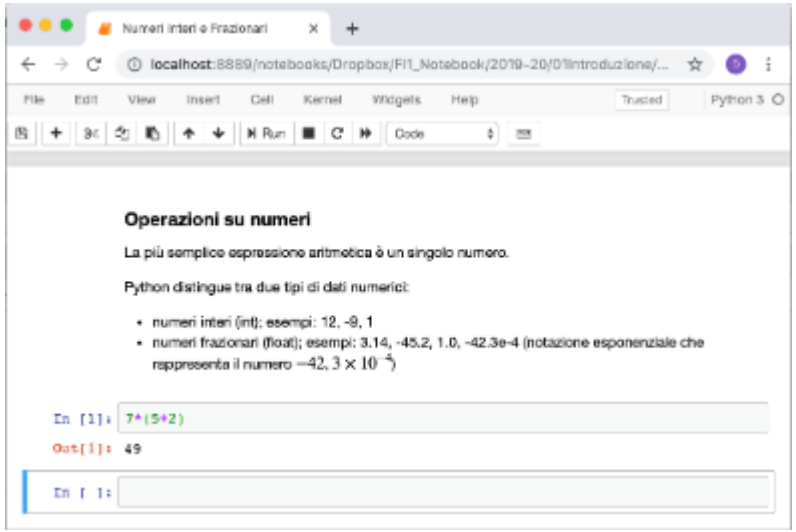
spyder | The Scientific Python Development Environment





Consultazione file Notebook su Python (distribuiti sul sito del corso)

Le note del corso su Python saranno distribuite nella forma di file Notebook (file .ipynb) che possono essere consultati con **Jupyter Lab**



jupyter

Le istruzioni relative all'istallazione degli ambienti di lavoro sono disponibili sul sito del corso

Sito Web del Corso

- Sulla piattaforma **Google Classroom**

https://classroom.google.com/c/MjUzMjM5OTQ1OTda?cjc=o7vjfgqu

- È necessario registrarsi al corso usando l'account Google Sapienza (username=vostra email istituzionale)
- Alternativamente potete accedere a classroom con l'account Google Sapienza dal link https://classroom.google.com
- Una volta effettuato il login su classroom, cliccate sul simbolo + in alto a destra
- Vi verrà chiesto il codice del corso. Inserite o7vjfgqu



INTRODUZIONE ALLA PROGRAMMAZIONE

Come usare correttamente gli LLM durante il corso

Uno strumento utile per capire, esercitarsi e imparare — non per saltare l'apprendimento

Materiale didattico per studentesse e studenti

Perché ne parliamo

1. **Gli LLM sono ormai ovunque**
   ChatGPT, Claude, Gemini, Copilot e simili sono strumenti che userete anche nel mondo del lavoro: negarli non ha senso.

2. **Ma la programmazione si impara facendola**
   Le competenze di base (sintassi, logica, debugging) si costruiscono solo scrivendo ed eseguendo codice in prima persona.

3. **L'obiettivo del corso non cambia**
   Dovete saper progettare, scrivere e correggere programmi da soli: è questo che verrà valutato all'esame.

Tre usi corretti dell'LLM

Nel corso, l'LLM va usato per capire e allenarsi — non per farsi risolvere gli esercizi



Spiegare il codice

Chiedere di illustrare, riga per riga, cosa fa un codice che non vi è chiaro

01



Generare varianti

Ottenere versioni alternative di un problema simile, per esercitarvi di più

02



Esempi e problemi

Farsi proporre nuovi esempi ed esercizi su cui allenarvi in autonomia

03

01



Farsi spiegare il codice

Usate l'LLM come un tutor che vi aiuta a capire, non come qualcuno che pensa al posto vostro.

Quando è utile

- Avete trovato online, in un libro o durante l'esercitazione un frammento di codice che non capite del tutto
- Volete sapere perché un vostro programma produce un certo output, riga per riga
- Volete un chiarimento su un costrutto (cicli, ricorsione, puntatori...) applicato a un esempio concreto
- State facendo debugging e volete capire il comportamento del codice prima di modificarlo

Prompt di esempio

"Spiegami riga per riga questo codice e dimmi perché il ciclo non si ferma quando i arriva a 10"

02

Generare varianti di un problema

Risolto un esercizio? Fatevi proporre una versione leggermente diversa e risolvetela di nuovo, da soli.

Come farlo bene

- Chiedete una variante che cambi dati, struttura o contesto ma mantenga la stessa logica risolutiva
- Provate a risolvere la variante voi, prima di guardare eventuali suggerimenti dell'LLM
- Usate le varianti per esercitarvi su uno stesso concetto (es. ricorsione, array, funzioni) finché non è automatico
- Fatevi generare una checklist di errori tipici da controllare nella vostra soluzione

Prompt di esempio

“Ho risolto l'esercizio sulla somma degli elementi di una lista. Proponimi una variante che differenzi tra valori positive e negativi, senza darmi la soluzione”

03



Farsi generare esempi e problemi

Un LLM è un generatore illimitato di esercizi su misura per il vostro livello e i vostri dubbi.

Cosa chiedere

- Esercizi mirati su un singolo argomento (es. “dammi 5 esercizi solo su cicli while”)
- Problemi di difficoltà crescente, per costruire gradualmente sicurezza
- Esempi guidati con soluzione commentata, da studiare prima di provare da soli
- Piccoli progetti che combinano più concetti visti a lezione

Prompt di esempio

“Dammi 3 esercizi di difficoltà crescente sull'uso delle liste, senza soluzione”

Cosa evitare

Un uso scorretto compromette proprio le competenze che l'esame verificherà

- Copiare la soluzione generata dall'LLM per un esercizio o un progetto da consegnare
- Chiedere all'LLM di risolvere un compito d'esame o una prova pratica **prima di aver provato a risolverla di persona**
- Usarlo per saltare la comprensione: incollare codice senza mai chiedersi perché funziona
- Farsi scrivere codice più avanzato di quanto sapreste scrivere voi stessi, senza poi saperlo spiegare

Fare da soli (fin dove si arriva)

Un uso scorretto compromette proprio le competenze che l'esame verificherà

- Per capire veramente un problema bisogna fare la fatica di risolverlo da soli. Questo sforzo ci permette di apprendere molto di più che leggere una soluzione fatta anche se commentata e spiegata

- Molti studi hanno già evidenziato le debolezze di un apprendimento fatto solo leggendo soluzioni pronte (calcolate da un LLM o altro). Si chiama «Cognitive Surrender» (resa cognitiva) e qui trovate un articolo scientifico (in inglese) che ne descrive gli effetti: https://osf.io/preprints/psyarxiv/yk25n_v1

La regola pratica

Se dopo aver usato l'LLM non siete in grado di riscrivere una soluzione simile da soli, a libro chiuso, allora non lo state usando bene.

- Prima ci provo da solo, poi chiedo aiuto
- Chiedo spiegazioni, non soluzioni pronte
- Uso l'LLM per esercitarmi di più, non di meno

In sintesi

Spiegare
per capire codice che non vi è chiaro

Variare
per esercitarvi su problemi simili

Generare
per allenarvi con nuovi esempi e problemi

L'obiettivo resta lo stesso: sapere programmare da soli, davanti a un compito o a un colloquio.

Introduzione:
L'elaborazione delle Informazioni

Elaborazione delle Informazioni

Un sistema per la rappresentazione e l'elaborazione di informazioni può

- raccogliere impressionanti quantità di dati
- elaborare i dati secondo le istruzioni fornite producendo nuovi dati
- rendere disponibili questi dati con prospettive diverse a utenti diversi e in parti diverse del mondo

Esempi: social networks, banca via internet, calcolo scientifico, video giochi, scacchi

Elaborazione delle Informazioni

- Computer (calcolatore, microprocessore…)
  - Esegue semplici operazioni aritmetiche e logiche a una velocità molto maggiore degli esseri umani (1 nanosecondo per operazione = 1 milionesimo di un millesimo di secondo, 10^{-9} secondi)
- Programma
  - Insieme di istruzioni eseguibili da un computer
- Hardware
  - Dispositivi fisici di un sistema di elaborazione
- Software
  - Programmi eseguiti su un elaboratore

Cos’è l’informatica?

L'informatica NON è lo studio dei computer come artefatti. Non studieremo come si costruisce un computer (o come lo si ripara)

Siamo interessati a USARE i computer

- L'informatica ha a che fare con lo studio di risoluzione di problemi (“problem solving”) per la cui soluzione si scrive un programma che viene eseguito da un computer

- L'informatica si basa su una serie di tradizioni intellettuali che comprende aspetti della matematica e dell'ingegneria

Cos’è l’informatica?

L'informatica svolge un ruolo sempre più importante in altre discipline:

- Biologia: analizzare il genoma umano
- Economia: la creazione di migliori modelli economici e finanziari
- Ambiente: i modelli climatici richiedono la moderna tecnologia informatica
- Letteratura: Analisi computerizzata aiuta a risolvere paternità contestata
- Cinema e intrattenimento: utilizzo di informatica grafica nei film e nei videogiochi
- Vita di tutti i giorni: sistemi di guida automatica, navigatori
- Intelligenza artificiale: classificazione automatica, conversazione con un utenti umani (ChatGPT), …

Programma e dati di ingresso

Un ***programma*** è una sequenza di istruzioni scritte in un linguaggio di programmazione (ed eseguibili da un calcolatore) che realizzano un algoritmo

Dati di ingresso*:* di solito un problema non deve essere risolto una sola volta ma molte volte con dati (di ingresso) diversi

Esempi:

- Trovare il percorso più breve in una rete stradale:
  dati di ingresso: rete stradale in esame, punto di partenza e punto di arrivo
- Eseguire una ricerca nel web:
  dati di ingresso: web (in quel momento), parole date nella ricerca

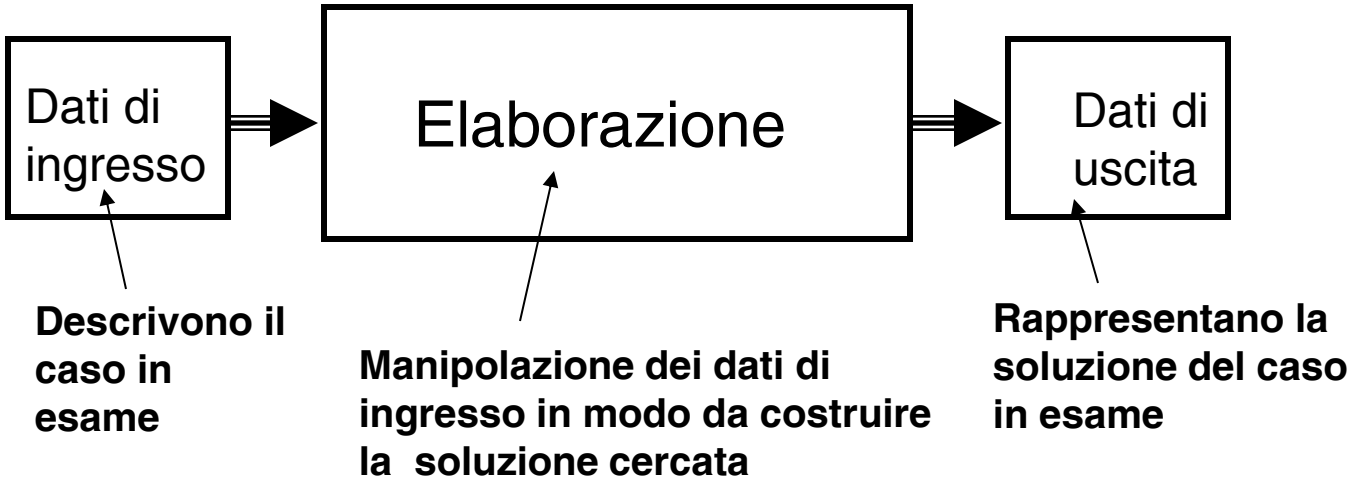
Elaborazione dell'informazione

Risoluzione di un problema

1. Dato un problema si trova un algoritmo di soluzione
2. Si scrive l'algoritmo di soluzione in un programma eseguibile da un calcolatore
3. Dato un programma, un calcolatore e dei dati di ingresso, l'esecuzione del programma con i dati di ingresso sul calcolatore risolve il problema su quei dati

Elaborazione dell'informazione

Come viene risolto un problema :



Algoritmi e Programmi

L'informatica studia la risoluzione di problemi (“problem solving”) realizzando un **programma** di soluzione che viene eseguito da un computer

Un **algoritmo** è come una “ricetta di cucina”, ovvero un procedimento schematico, composto da una sequenza di istruzioni elementari, che consente di risolvere un problema

Esempi di problemi: ordinare alfabeticamente una serie di nomi, trovare le pagine più interessanti in una ricerca su un motore di ricerca, trovare il percorso stradale più breve

Algoritmo

In modo informale, si può pensare a un algoritmo come ad una procedura per risolvere un problema

Una definizione più formale richiede che:

- I passi (le **istruzioni**) dell'algoritmo siano definiti in modo chiaro e **non ambiguo**
- Sia eseguibile, nel senso che i suoi passi siano **eseguibili da una macchina**
- Sia finito, nel senso che **termini dopo un numero finito di passi**

Riassumendo:

problema ➔ algoritmo ➔ programma

Individuare un algoritmo per un dato problema può essere molto complesso: Conviene operare per **livelli di astrazione**:

- si parte da una versione molto generale
- si **raffinano** via via le varie parti dell'algoritmo fino ad ottenere una descrizione dettagliata che può essere direttamente codificata in un linguaggio di programmazione

→ metodo dei raffinamenti successivi

Pseudocodice per la specifica di algoritmi

- si usano **frasi in linguaggio naturale** che esprimono operazioni o condizioni
- operazioni vengono eseguite in sequenza, tranne che per le . . .
- **strutture di controllo** dell'esecuzione delle operazioni (specificate usando parole chiave)

if condizione
then do operazione 1
else do operazione 2

for ogni valore compreso tra . . . e . . .
do operazione

while condizione
do operazione

do operazione
while condizione

Lo pseudocodice si presta bene al metodo dei raffinamenti successivi: un **raffinamento** consiste nel sostituire un'operazione con

- una sequenza di operazioni, oppure
- una struttura di controllo

Esempio: Calcolo del massimo comune divisore di due interi positivi m ed n

Algoritmo

1. calcola l'insieme A dei divisori di m
2. calcola l'insieme B dei divisori di n
3. calcola l'insieme C intersezione di A e B (divisori comuni)
4. restituisci il valore massimo tra quelli in C

Raffinamento del passo 1

1.1 inizializza A all'insieme vuoto
1.2 **for** ogni numero i compreso tra 1 ed m
if i è divisore di m
then aggiungi i ad A

Raffinamento del passo 2

Identico al raffinamento del passo 1

…

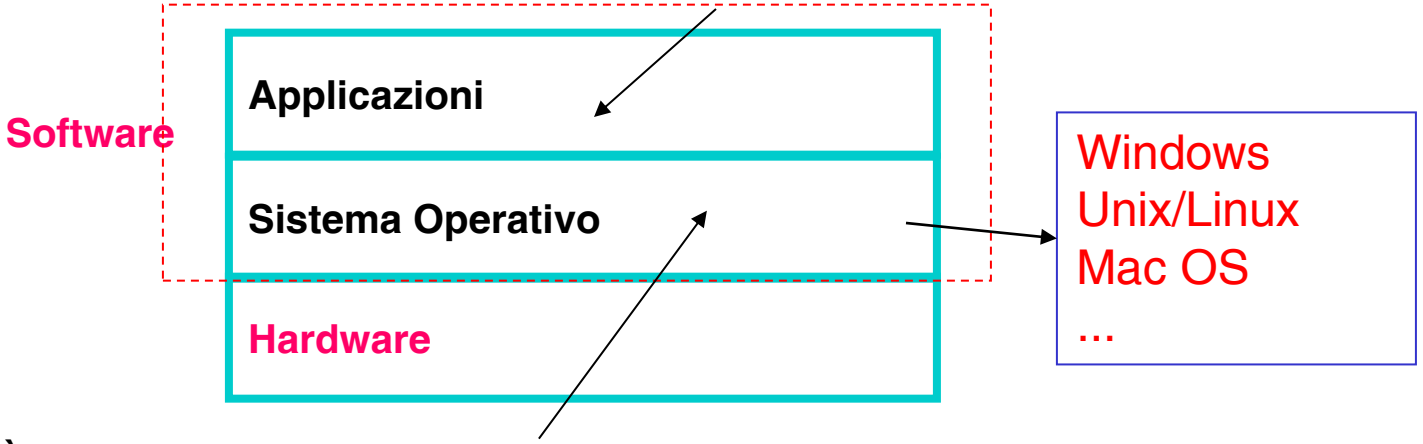
Programmi: rappresentano algoritmi in un linguaggio di programmazione

Esempio: Programma che calcola il MCD tra 2 interi

```
m=1457
n=62
A=set()
B=set()
for i in range(1,m+1):
    if m % i == 0:   #il resto della divisione tra m e i vale zero
        A.add(i)
for i in range(1,n+1):
    if n % i == 0:
        B.add(i)
C = A & B           #  & calcola l'intersezione
print(max(C))
```

Applicazioni e Sistema Operativo

Applicazioni: È il livello di SW con cui interagisce l'utente e comprende programmi quali: *Word, PowerPoint, Excel, Chrome, …..*



È il livello di SW che interagisce direttamente con l'HW e che si occupa di un uso corretto ed efficiente delle risorse fisiche

Linguaggi di programmazione

Dato un problema, un algoritmo di soluzione deve essere scritto in un linguaggio comprensibile da una macchina

I calcolatori sono in grado di comprendere **solo** programmi scritti in ***linguaggio macchina***, che è un linguaggio specifico per il processore (CPU) del calcolatore e complicato per gli uomini

I programmi sono scritti in un linguaggio più facile da utilizzare; ce ne sono tantissimi che sono detti ***linguaggi ad alto livello***; Python è uno di questi, semplice da imparare e quindi adatto per iniziare; ce ne sono ovviamente molti altri...

Linguaggi di programmazione

Come già visto ci sono molti linguaggi di programmazione: Pascal, C, C++, Java, Ada, Perl, Php, Python…

- Questi linguaggi condividono gli stessi principi
- Ognuno però ha caratteristiche specifiche che lo distinguono dagli altri

Nelle applicazioni di oggi

- Elaborazioni complesse dei dati sono spesso fatte usando linguaggi **diversi**
- Imparando i concetti fondamentali di **un linguaggio** sarete in grado di apprendere velocemente da soli nuovi linguaggi

Linguaggi di programmazione

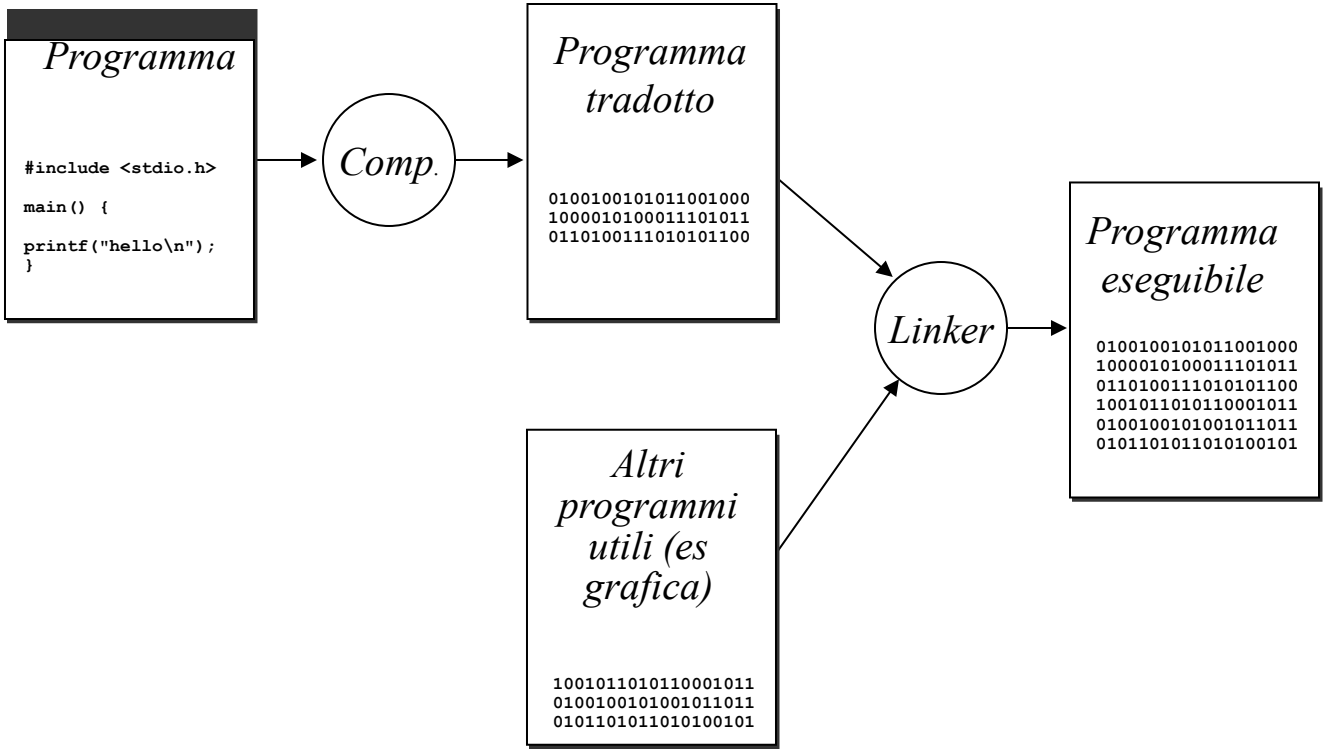
I linguaggi ad alto livello **non sono** direttamente eseguibili da un elaboratore

Per eseguire un programma scritto in un linguaggio ad alto livello dobbiamo ***tradurlo*** nel linguaggio macchina

Due possibilità principali

- **Compilatore**: programma che traduce un linguaggio ad alto livello in un linguaggio macchina eseguibile dal sistema operativo
- **Interprete:** il programma scritto in linguaggio ad alto livello viene simulato senza tradurlo completamente (una istruzione alla volta)

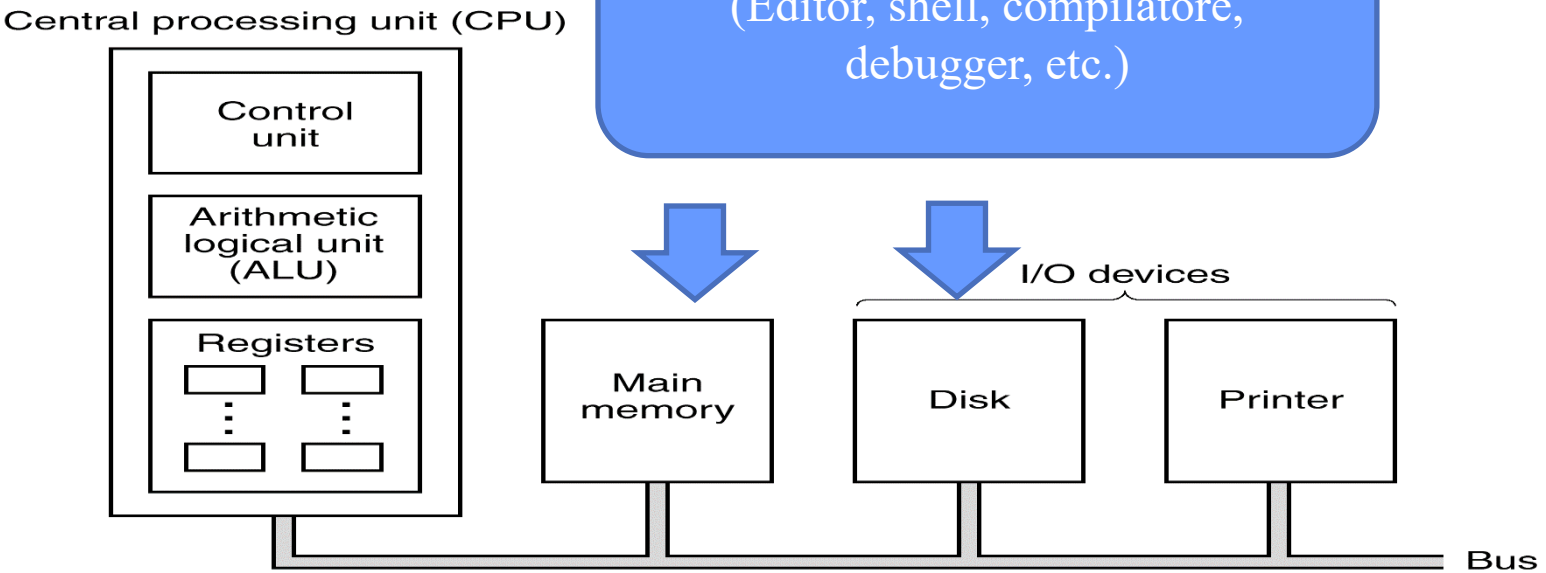
Il processo di compilazione



Ambienti di programmazione



Ambiente di programmazione
(Editor, shell, compilatore,
debugger, etc.)



Introduzione:
Il linguaggio Python

Python

- Sviluppato all'inizio degli Anni Novanta da Guido van Rossum
  - Il suo nome fu scelto per via della passione dell'autore per i Monty Python e per la loro serie televisiva Monty Python's Flying Circus (BBC, 1969 - 1974)
- L'intento era quello di progettare un linguaggio che consentisse di operare agevolmente con dati complessi e articolati:
  - Dinamicità
  - Semplicità
  - Flessibilità



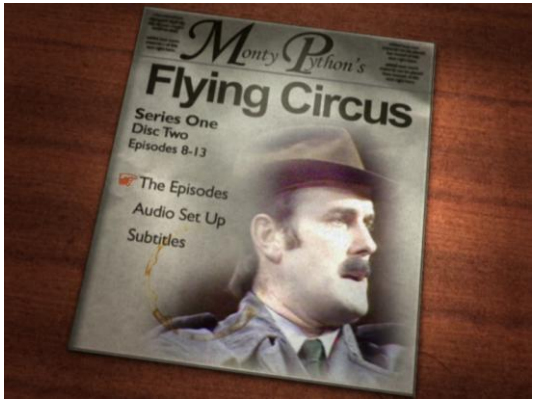


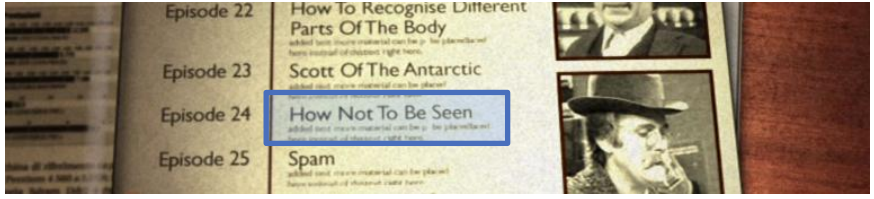


Monty Python ?

- Comici inglesi
- Monty Python's Flying Circus
- BBC 1969-74
- Comicità grottesca e surreale (inventori della parola Spam)
  - Graham Chapman
  - John Cleese
  - Terry Gilliam
  - **Eric Idle → editor Python**
  - Terry Jones
  - Michael Palin











Python

- Python è **free software**: il download dell'interprete e l'uso di Python è completamente gratuito, ma oltre a questo Python può essere liberamente modificato e ridistribuito, secondo le regole di una licenza pienamente *open-source*
- Lo sviluppo di Python viene coordinato dall'organizzazione no-profit Python Software Foundation
- Guido van Rossum è rimasto per anni il *“Benevolo dittatore a vita”* di Python, colui che all’interno della comunità di sviluppatori avrebbe comunque avuto il peso maggiore nelle decisioni relative allo sviluppo del linguaggio
- Dal 2018 van Rossum ha però lasciato il suo ruolo, per via delle crescenti difficoltà legate al suo svolgimento. Ha, in particolare, dichiarato:
- *"I don't ever want to have to fight so hard for a PEP (ndr: proposta di modifica al linguaggio) and find that so many people despise (sic) my decisions."*

Python

- Supporta **diversi paradigmi di programmazione**:
  - Imperativo (enfasi sulle operazioni intese come **comandi** che cambiano lo stato dell'elaborazione)
  - Funzionale (enfasi sulle operazioni intese come **funzioni** che calcolano risultati)
  - Orientato agli oggetti (enfasi sugli **oggetti** che complessivamente rappresentano il dominio di interesse)
- Offre un controllo dei tipi
  - Forte (strong typing): Ogni elemento sintattico che denota un valore (ad es. variabile) è associato a un determinato tipo durante l'esecuzione
  - Eseguito a run-time (dynamic typing): il controllo del tipo della variabile è effettuato a runtime

Python

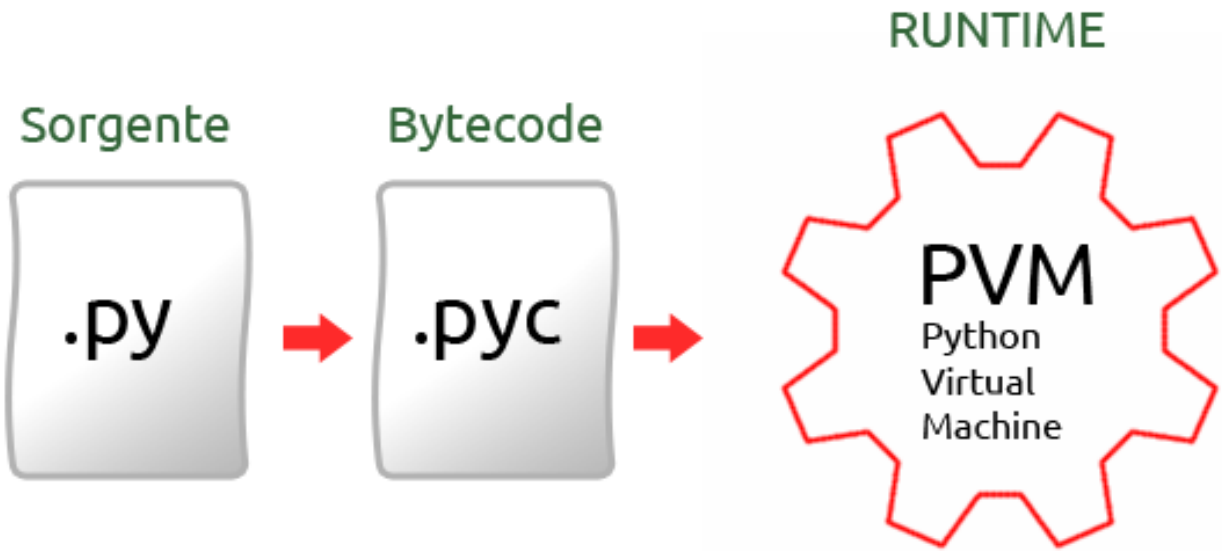
- Offre una gestione automatica della memoria
  - Garbage collector
- Fornito di una libreria built-in estremamente ricca
- Sono disponibili numerosissime altre librerie
- Offre costrutti robusti per la gestione delle eccezioni
  - come mezzo per segnalare e controllare eventuali condizioni di errore (incluse le eccezioni generate dagli errori di sintassi)

Python

Linguaggio interpretato (o meglio pseudo-compilato):

- un interprete si occupa di analizzare il codice sorgente (semplici file testuali con estensione .py) e, se sintatticamente corretto, di eseguirlo
- il codice sorgente passa prima per una fase di pre-compilazione in bytecode (riduce la dipendenza dall'hardware)
  - viene riutilizzato dopo la prima esecuzione del programma, evitando così di dover ogni volta interpretare il sorgente e incrementando di conseguenza le prestazioni (in modo trasparente per l'utente)
- non esiste una fase di compilazione separata che generi un file eseguibile partendo dal sorgente
  - il codice sorgente non viene convertito direttamente in linguaggio macchina, ma il bytecode prodotto viene eseguito da una **macchina virtuale Python (PVM)**
- Portabilità
- Indipendenza dalla piattaforma
- Java segue grossomodo la stessa filosofia

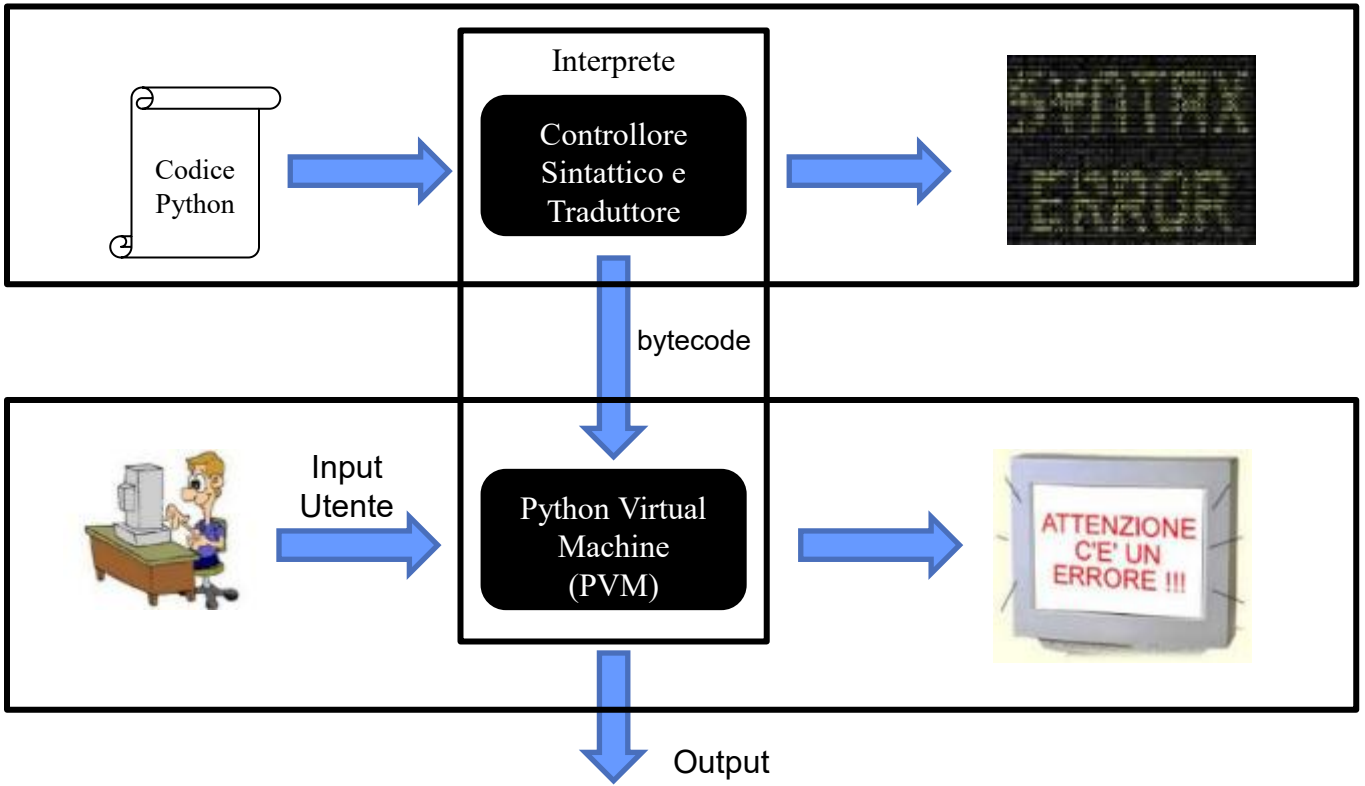
Interpretazione di un programma Python (1\2)



Macchina reale: HW+ Sistema operativo

Una macchina virtuale è un **programma** per una macchina reale (cpu+hw+sistema operativo) che simula il comportamento di una macchina reale

Interpretazione di un programma Python (2\2)



Esecuzione di script/programmi *.py*

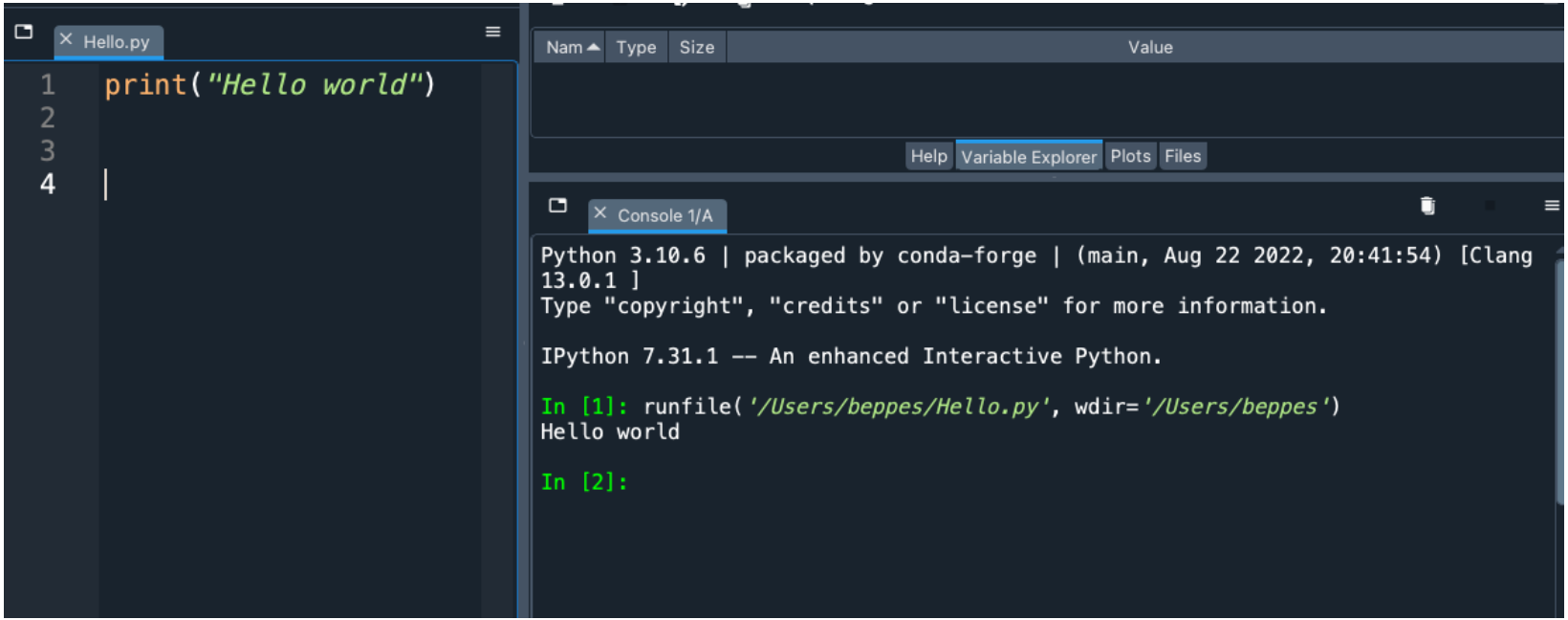
Per prima cosa bisogna generare un semplice file di testo e salvarlo con estensione **.py**, per esempio, **helloworld.py:**

```
print('Hello World!')
```

Il modo più agevole per creare uno script python è usare un **Editor per programmi Python** (come ad esempio Spyder – vedi oltre)

Come primo esempio, abbiamo inserito nel file **hello.py** una sola riga di codice, costituita dalla funzione print(), che come risultato stamperà una stringa

Eseguendo il file (run) avremo il seguente risultato:



Modalità Interattiva

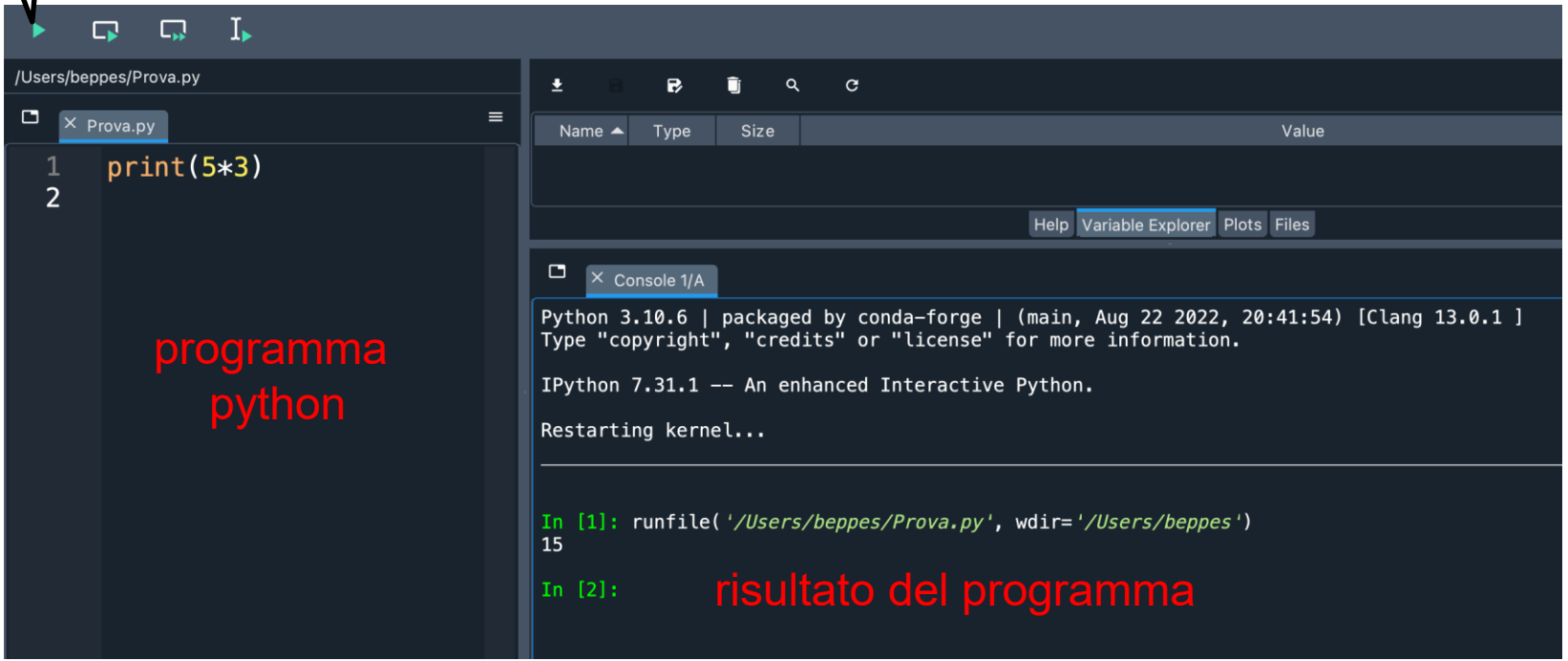
- In ogni installazione di Python è disponibile anche un interprete interattivo in grado di leggere e valutare man mano le espressioni inserite dall'utente



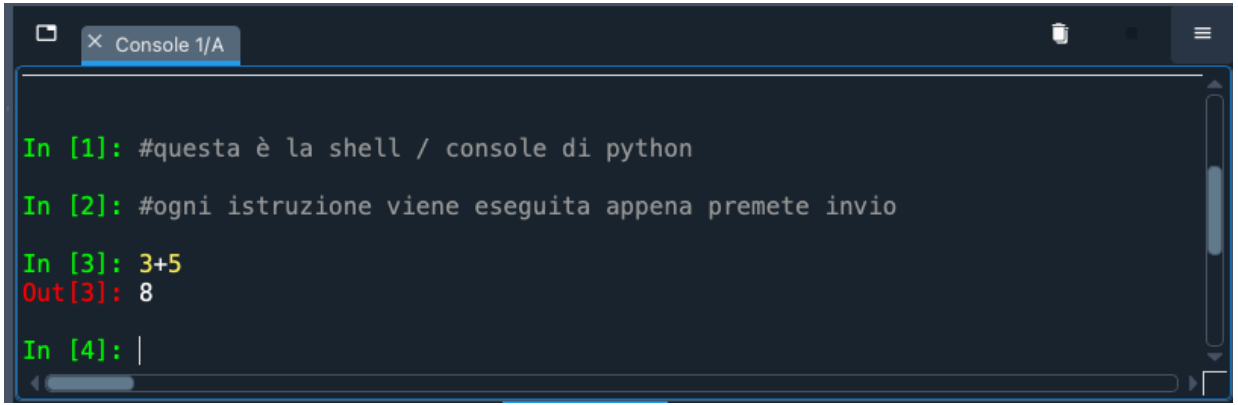
- N.B. In uno script Python per vedere il valore di una espressione o il contenuto di una variabile occorre usare una print:



Ambiente di sviluppo Python Spyder



Python: Shell o Editor?




```
1  #questo è l'editor di python
2
3  #tutte le istruziojni vengono eseguite insieme con il comando run
4  #le ricghe che cominciano con # sono commenti
5  #per mostrare il risultato delle espressioni occorre una print()
6
7  print(5+3)
8
9
```

```
8
```

Python IDLE



IDLE :
Integrated DeveLopment Environment

Python 3.5.0 Shell

```
>>>
>>>
>>>
>>>
>>>
>>> #questa è la shell di Python
>>> #ogni istruzione viene eseguita appena premete invio
>>> #fate caso ai colori
>>> # e controllate lo spazio dopo >>>
>>> 3+5
8
>>> "ciao"
'ciao'
>>>
```

02_Esempio editor.py - /Users/beppes/Dropbox/doc_dida/corsopython/Lezioni_2015-2016/Lezione_01/02_Esempio editor.py (3.5.0)

```
#questo è l'editor di file di Python
#tutte le istruzioni vengono eseguite insieme (F5 o Run->Run module)
#tutte le righe che cominciano con # vengono ignorate (commenti)
#NON mostra il risultato delle espressioni --> per vedere il risultato DOVETE usare print
#fate caso ai colori

print("Ciao")
print(5+3)
8+8
```

Ln: 10 Col: 0

F5 ->

Python 3.5.0 Shell

```
Python 3.5.0 (v3.5.0:374f501f4567, Sep 12 2015, 11:00:19)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>>
 RESTART: /Users/beppes/Dropbox/doc_dida/corsopython/Lezio
ni_2015-2016/Lezione_01/02_Esempio editor.py
Ciao
8
>>>
```

Ln: 8 Col: 4